

Object-Oriented Programming

Lecture 3: Python Basics II

Prof. Jaehyung Seo

(seojae777@konkuk.ac.kr)

Language Intelligence & Representation Lab.

Python Data Structures

Beyond Basic Containers

- Moving from simple syntax to architectural choices; understanding when to use built-in structures versus custom classes.

Immutability vs. Mutability

- Analyzing the design implications of tuples and frozen dataclasses versus lists and dictionaries regarding state management and hashing.

Modern Python Features

- Leveraging typing.NamedTuple and @dataclasses to reduce boilerplate and enforce type hinting in object definitions.

Performance & Complexity

- Evaluating time complexity ($O(1)$ vs $O(n)$) for lookups and understanding the memory overhead of different object types.

Overview Code Example

Code Snippet:

The spectrum of data structures covered in Lecture 2

```
from typing import NamedTuple
from dataclasses import dataclass
```

1. Immutable Record: Efficient, hashable, no behavior

```
class PointTuple(NamedTuple):
    x: int
    y: int
```

2. Mutable Data Object: Auto-generated methods, mutable

```
@dataclass
class PointClass:
    x: int
    y: int
```

Empty Objects

The Base `object()` and Empty Classes

The Base Object Limitation

- Instances of the built-in `object()` class are stateless and memory-optimized; they lack a `__dict__` attribute and cannot accept dynamic attributes

Custom Empty Classes

- To create a dynamic "bag" of attributes, one must define a subclass (e.g., `class Empty: pass`), which initializes the `__dict__` dictionary for state storage

Memory Implications

- Using generic objects for data storage is often inefficient compared to `slots` or specialized structures like tuples due to the overhead of the instance dictionary.

Empty Objects

Code Snippet:

```
# 1. The built-in object is "closed" to new attributes
o = object()
# o.x = 5 # Raises AttributeError: 'object' object has no attribute 'x'

# 2. A custom empty class allows dynamic attribute assignment
class Empty:
    pass

e = Empty()
e.x = 5      # Valid: stored in e.__dict__
e.y = "data"
```

Tuples and Named Tuples – 1

Semantic Structure

- Tuples are best understood as **records** (structs) where position indicates meaning (e.g., x-coordinate at index 0), rather than just read-only lists

Immutability & Hashing

- Because tuples are immutable, they are hashable (if their contents are hashable), making them valid keys for dictionaries and elements for sets

Memory Efficiency

- Tuples consume less memory than lists and are faster to create, making them ideal for fixed collections of heterogeneous data

Tuples Code Example

Code Snippet:

```
# A tuple representing a stock record (Symbol, Current, High, Low)
stock = ("GOOG", 613.30, 625.86, 610.50)

# Unpacking for readability (replaces index access)
symbol, current, high, low = stock

# Tuples are immutable
stock[7] = 615.00 # Raises TypeError: 'tuple' object does not support
item assignment

# Valid usage as a dictionary key
portfolio = {stock: 10}
```

Tuples and Named Tuples – 2

`collections.namedtuple` – Readable Records

Solving the "Magic Index" Problem

- Standard tuples rely on integer indices (e.g., stock), which reduces code readability. `namedtuple` assigns meaningful identifiers to positions

Factory Function

- `collections.namedtuple` is a factory function that generates a new subclass of `tuple` with named fields

Backward Compatibility

- Named tuples behave exactly like standard tuples; they are iterable, unpackable, and indexable, ensuring compatibility with existing codebases

Collections.namedtuple code example

Code Snippet:

```
from collections import namedtuple

# Create a new class called 'Stock' with specific fields
Stock = namedtuple("Stock", ["symbol", "current", "high",
                              "low"])

# Create an instance
s = Stock("GOOG", 613.30, high=625.86, low=610.50)

# Access by name (Easy to read)
print(s.high)    # 625.86

# Access by index (Works like a regular tuple)
print(s[2])      # 625.86
```

Named Tuples via `typing.NamedTuple`

Modern Named Tuples with `typing.NamedTuple`

Type Hint Integration

- Introduced in Python 3.6, `typing.NamedTuple` allows defining named tuples using modern class syntax with type annotations

Declarative Syntax

- This approach is more readable than the `collections` factory function strings, resembling a standard class definition

Same Underlying Mechanism

- Despite the syntactic difference, it still produces a tuple subclass, retaining performance benefits and immutability

typing.NamedTuple Code Example

Code Snippet:

```
from typing import NamedTuple

class Stock(NamedTuple):
    symbol: str
    current: float
    high: float
    low: float

    # You can add methods!
    @property
    def spread(self) -> float:
        return self.high - self.low

s = Stock("GOOG", 613.30, 625.86, 610.50)
print(f"Spread: {s.spread:.2f}") # Output: Spread: 15.36
```

Introduction to Dataclasses

Boilerplate Reduction

- The `@dataclass` decorator (Python 3.7+) automatically generates `__init__`, `__repr__`, `__eq__`, and other dunder methods based on type annotations

Mutable by Default

- Unlike NamedTuples, `Dataclasses` are mutable, making them suitable for objects that need to change state over time

Readability

- They provide a clean, declarative way to define objects that exist primarily to store data, removing the repetitive code of manual class definitions

Dataclasses Code Example

Code Snippet:

```
from dataclasses import dataclass

@dataclass
class Stock:
    symbol: str
    current: float
    high: float
    low: float

# __init__ is auto-generated
s = Stock("GOOG", 613.30, 625.86, 610.50)

# __repr__ is auto-generated
print(s)
# Output: Stock(symbol='GOOG', current=613.3, high=625.86, low=610.5)

s.current = 614.50  # Valid: Dataclasses are mutable
```

Immutable Dataclasses (Frozen)

Enforcing Immutability

- Setting `@dataclass(frozen=True)` makes instances immutable, preventing attribute modification after initialization

Hashability

- Frozen dataclasses generate a `__hash__` method, allowing them to be used as dictionary keys or set elements (unlike standard mutable dataclasses)

Thread Safety

- Immutable data structures are inherently thread-safe for reading, which is beneficial in concurrent applications

Frozen Dataclass Code Example

Code Snippet:

```
@dataclass(frozen=True)
class ImmutableStock:
    symbol: str
    current: float

s = ImmutableStock("MSFT", 305.00)
# s.current = 310.00 # Raises FrozenInstanceError

# Can be used as a dict key because it is hashable
# A regular dataclass would raise "TypeError: unhashable type"
portfolio = {s: 100}
print(portfolio[s]) # Output: 100
```

Advanced Dataclass Features (Ordering)

Automatic Comparisons

- Using `order=True` generates comparison methods (`__lt__`, `__gt__`, etc.), allowing instances to be sorted naturally

Field Ordering

- Comparisons are performed sequentially based on the order fields are defined (e.g., first compares `symbol`, then `current`)

Default Values

- Fields can have default values, simplifying initialization for optional data

Ordering Dataclass Code Example

Code Snippet:

```
@dataclass(order=True)
class Stock:
    # Sorts primarily by symbol, then by current price
    symbol: str
    current: float = 0.0 # Default value

s1 = Stock("AAPL", 150.0)
s2 = Stock("GOOG", 2800.0)
s3 = Stock("AAPL", 145.0)

# Automatic comparison logic enabled by order=True
print(s1 < s2) # True ("AAPL" comes before "GOOG")
print(s3 < s1) # True ("AAPL" same, 145.0 < 150.0)

stocks = [s1, s2, s3]
stocks.sort() # Sorts automatically
```

Dictionaries – 1

Dictionaries – High-Performance Mappings

Hash Table Implementation

- Dictionaries use hash tables to map keys to values, offering $O(1)$ average time complexity for lookups, insertions, and deletions

Key Constraints

- Keys must be **hashable** (immutable types like strings, numbers, tuples, or frozen dataclasses). Values can be any object

Insertion Order

- Since Python 3.7, dictionaries are guaranteed to preserve the insertion order of keys, a feature that can be relied upon for design

Dictionaries – 2

Iteration and Views

View Objects

- Methods like `.keys()`, `.values()`, and `.items()` return "views"
- Dynamic windows into the dictionary that update instantly if the dictionary changes

Efficient Iteration

- Iterating directly over a dictionary yields keys. Using `.items()` allows for tuple unpacking of key-value pairs in a loop

Membership Testing

- Checking `key in dict` is an $O(1)$ operation, significantly faster than searching a list

Dictionary Views Code Example

Code Snippet:

```
portfolio = {"AAPL": 10, "GOOG": 5}

# .items() returns a dynamic view of key-value pairs
for symbol, shares in portfolio.items():
    print(f"{symbol}: {shares}")

# Dynamic view reflection
keys_view = portfolio.keys()
print(f"Before: {keys_view}") # dict_keys(['AAPL', 'GOOG'])

portfolio["MSFT"] = 100

# The view updates automatically without re-calling .keys()
print(f"After: {keys_view}") # dict_keys(['AAPL', 'GOOG', 'MSFT'])
```

Dictionary Use Cases

Dictionary Design Patterns

Homogeneous Collection

- Used as an index mapping keys to values of the same type (e.g., stock symbols to prices). This mimics a database table with a primary key

Heterogeneous Record

- Used to store different attributes of a single entity (e.g., {'name': 'Google', 'price': 1200}). While flexible, `NamedTuple` or `dataclass` is often preferred for type safety

TypedDict

- Python 3.8+ introduces `TypedDict` to enforce type hints on dictionary keys and values, bridging the gap between flexible dictionaries and structured classes

Dictionary Use Cases Code

Code Snippet:

```
from typing import TypedDict

# Use Case 1: Index (Homogeneous) - Symbol maps to Price
stocks: dict[str, float] = {"GOOG": 1235.20, "MSFT": 110.41}

# Use Case 2: Record (Heterogeneous) - Different types
class StockRecord(TypedDict):
    name: str
    shares: int

# Static type checkers will warn if you use wrong keys
record: StockRecord = {"name": "GOOG", "shares": 100}
# record["price"] = 1200 # Error: 'price' is not in StockRecord
```

Using defaultdict

Simplifying Logic with defaultdict

Automatic Initialization

- defaultdict takes a factory function (e.g., int, list) to create values for missing keys automatically, eliminating repetitious if key not in dict checks

Cleaner Code

- It replaces the verbose .setdefault() method or exception handling logic when building complex structures like grouped lists or frequency counters.

Type Hinting

- When using type hints, explicit definition (e.g., defaultdict[str, int]) clarifies the expected key and value types for static analysis.

defaultdict Code

Code Snippet:

```
from collections import defaultdict

# Tell defaultdict to use 'list' for missing keys
grouped_emails = defaultdict(list)

emails = [("work", "boss@corp.com"), ("home", "mom@gmail.com")]

for tag, email in emails:
    # If 'tag' is new, a new list is created automatically.
    # We can just append immediately.
    grouped_emails[tag].append(email)

print(grouped_emails)
# Output: defaultdict(<class 'list'>, {'work': ['boss@corp.com'], ...})
```


Counter

Counting with collections.Counter

Specialized Dictionary

- `Counter` is a dictionary subclass specifically designed for counting hashable objects, defaulting missing counts to zero

Statistical Methods

- It provides methods like `most_common(n)` to easily retrieve top-frequency items, which would require manual sorting with a standard dictionary.

Multiset Behavior

- Counters act like multisets (bags), supporting mathematical operations like addition, subtraction, union, and intersection on counts.

Counter Code

Code Snippet:

```
from collections import Counter

responses = ["yes", "no", "yes", "maybe", "no", "yes"]
counts = Counter(responses)

print(counts["yes"])           # Output: 3
print(counts.most_common(1))  # Output: [('yes', 3)]

# Missing keys return 0, not KeyError
print(counts["unknown"])      # Output: 0

# Math with counters
c1 = Counter(a=3, b=1)
c2 = Counter(a=1, b=2)
print(c1 + c2)                # Output: Counter({'a': 4, 'b': 3})
```

Lists

Lists – The Versatile Sequence

Usage Context

- Lists should generally store homogeneous data (instances of the same type) where order matters. Using lists for heterogeneous attributes (e.g., [name, age]) is considered anti-pattern.

Mutability

- Lists are mutable sequences, supporting dynamic insertion, deletion, and replacement of elements.

Performance

- Implemented as dynamic arrays, they offer $O(1)$ append operations (amortized) but $O(n)$ for insertions/deletions at the start of the list.

Lists Code

Code Snippet:

```
# Homogeneous list: correct usage (all floats)
prices: list[float] = [10.5, 20.0, 15.5]

# List comprehension is Pythonic and fast
taxed_prices = [p * 1.08 for p in prices]

# Heterogeneous list: discouraged
person = ["John", 30, "Engineer"]

# Performance note
data = list(range(100000))
data.append(99)      # Fast:  $O(1)$  - Instant
data.insert(0, 99)   # Slow:  $O(n)$  - Must shift 100,000 items
```

Sorting Lists

Sorting Mechanisms

Default Ordering

- `sort()` uses standard comparison operators (`<`, `>`). For strings, this is lexicographical; for numbers, numerical

Custom Objects

- To sort custom classes, implement the `__lt__` (less than) magic method or use a `key` function

Key Functions

- The `key` argument injects external sorting logic, allowing clean separation of concerns

Sorting Lists Code

Code Snippet:

```
from operator import attrgetter

class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

products = [Product("Widget", 100), Product("Gadget", 50)]

# Sort by price using a lambda function
products.sort(key=lambda p: p.price)

# Sort by name using attrgetter (faster and cleaner)
products.sort(key=attrgetter("name"))
```

Sets

Sets – Unique and Unordered

Uniqueness

- Sets guarantee unique elements. Converting a list to a `set` is the idiomatic Python way to deduplicate data

Hashing Requirement

- Like dictionary keys, elements in a set must be **hashable** (immutable)

Performance

- Membership testing (`x in s`) is $O(1)$ average, making sets significantly faster than lists ($O(n)$) for lookups

Sets Code

Code Snippet:

```
# 1. Deduplication
raw_ids = [101, 102, 103, 101, 102]
unique_ids = set(raw_ids) # {101, 102, 103}

# 2. Set Algebra
admins = {"Alice", "Bob"}
users = {"Bob", "Charlie", "Dave"}

both = admins & users # Intersection: {"Bob"}
regular_users = users - admins # {"Charlie", "Dave"}

# Fast lookup (O(1))
if "Alice" in admins:
    print("Access Granted")
```


Queues

FIFO, LIFO, and Priority Queues

Limitations of List

- Using `list.pop(0)` for a queue is $O(n)$ because all elements shift. Inefficient for large datasets

deque

- `collections.deque` (Double-Ended Queue) is optimized for $O(1)$ appends and pops from both ends

Priority Queues

- `heapq` or `queue.PriorityQueue` allows retrieving items based on importance rather than insertion order

Queues Code

Code Snippet:

```
from collections import deque
from queue import PriorityQueue

# 1. Deque: Fast FIFO
dq = deque()
dq.append("Task 1")
dq.popleft() # Fast O(1). List pop(0) is slow O(n)

# 2. Priority Queue
pq = PriorityQueue()
pq.put((2, "Write Code"))
pq.put((1, "Fix Critical Bug"))

next_task = pq.get()
print(next_task) # (1, 'Fix Critical Bug')
```

Case Study

Case Study – K-Nearest Neighbors

Problem Domain

- Building a machine learning classifier (k-NN) to classify data samples based on physical measurements

Design Goal

- Create a robust, memory-efficient `Sample` object and a `TrainingData` container

Decision Matrix

- We evaluate `dict` vs `list` vs `class`. The data is immutable once recorded, leading to specific structure choices

Case Study Code

Code Snippet:

```
from dataclasses import dataclass

# Chosen Implementation: Frozen Dataclass
@dataclass(frozen=True)
class Sample:
    sepal_len: float
    sepal_width: float
    petal_len: float
    petal_width: float
    species: str

s1 = Sample(5.1, 3.5, 1.4, 0.2, "Iris-setosa")
# s1.sepal_len = 5.2 # Error: FrozenInstanceError

training_set = {s1} # Hashable: can be used in sets
```

Conclusion

Choosing the Right Structure

Intent Driven

- Use `tuples/NamedTuples` for immutable records, `lists` for ordered collections, `sets` for uniqueness, and `dicts` for lookups

Performance

- Built-in structures (`set`, `dict`) are highly optimized C implementations. Prefer them over custom logic

Modern Python

- `Dataclasses` and `TypedDict` modernize how we structure data, reducing boilerplate while maintaining OOP principles

Recall

Key Concepts Review

Empty Objects

- object() is stateless; custom empty classes allow dynamic attributes but lack structure

Dataclasses

- Automate class boilerplate; `frozen=True` mimics immutability

Dictionary vs. Object

- Dictionaries are for dynamic mappings; Objects are for fixed schemas with behavior

Complexity

- Dictionary/Set lookup is $O(1)$; List lookup is $O(n)$

Exercises

Applying Structure Choices

1. Refactor

- Take a previous script using parallel lists (e.g., names, ages) and refactor it to use a list of `dataclass` objects

2. Performance Test

- Compare lookup speed of finding a user ID in a `list` of 1,000,000 items versus a `set` using `timeit`

3. API Design

- Implement a generic `Stack` class using a list, then refactor to `collections.deque` and compare the API surface

Summary

Data Structures in OOP

Data Structures are Objects

- In Python, lists, dicts, and sets are full-fledged objects with methods and behaviors

Specialization

- Specialized structures (Counter, deque, defaultdict) simplify code and improve performance over generic types

Architectural Fit

- Choose structures based on data characteristics (uniqueness, order, mutability) to enforce design constraints naturally